Sprint 11 · Visión por computador

¿podemos aprovechar las imágenes del cliente?

FECHAS
17–20 de noviembre

HORAS
17 h (5 h teoría · 7 h práctica · 5 h PBL) · **Módulo 4, UD 4.4–4.6**

Qué aprenderás

1. Cómo "ve" una red: imágenes como matrices, convolución, *pooling* y arquitectura de una CNN.
2. Entrenar una CNN pequeña y mejorar su generalización con *data augmentation*.
3. Reutilizar redes preentrenadas: *transfer learning* y *fine-tuning*.
4. Detección de objetos y segmentación: IoU, NMS, AP/mAP y un detector clásico con OpenCV.
5. Evaluar un clasificador de imágenes con métricas por clase y **análisis de errores**.

Requisitos previos

- Sprint 10 (redes neuronales, Keras, curvas de aprendizaje, regularización).
- Todo funciona **sin internet** con imágenes sintéticas (dibujos generados por código, no fotos reales).
- Algunas celdas opcionales para Colab (descarga de `tf_flowers`, pesos de MobileNetV2 con ImageNet, YOLO) están marcadas como **«requiere internet»**: en el taller no se ejecutan (llevan un interruptor `USAR_INTERNET = False`).

Prácticas

Notebook	Tema	Tiempo
`S11_01_cnn`	Convolución, pooling y CNN en Keras (dígitos 8×8); *data augmentation*	1 h 45
`S11_02_transfer_learning`	Transfer learning y fine-tuning (con preentrenamiento propio y MobileNetV2 en Colab)	1 h 45
`S11_03_deteccion_segmentacion`	IoU, NMS, mAP, detector con OpenCV, segmentación (y YOLO opcional)	1 h 45
`S11_04_reto_vision_cliente`	Esqueleto del reto: dataset por carpetas, transfer learning, análisis de errores	1 h 45 (+ trabajo PBL)

Material extra: `generar_dataset_sintetico.py` crea el mini dataset de tipos de alojamiento (una carpeta por clase) para probar el flujo. No se sube al repositorio: se genera con el script (`python generar_dataset_sintetico.py`).

RETO Reto PBL · «Visión artificial para el cliente»

En pocas palabras (léelo primero)

Qué os pide el cliente. TurisData recibe **cientos de fotos** de alojamientos y zonas cada semana, y una persona las clasifica a mano (hotel, apartamentos, casa rural, camping, piscina). Quiere saber si un modelo puede **pre-clasificarlas** y, sobre todo, **en qué se equivoca**, para decidir cuánto puede fiarse y **qué fotos debe revisar una persona**.

Lo importante del reto no es el acierto, sino saber cuándo no fiarse. Un modelo que acierta el 90 % pero no sabe cuándo falla es peligroso; uno que acierta el 85 % y dice «esto no lo tengo claro, que lo revise una persona» es útil.

Qué vais a hacer, en seis pasos:

1. **Preparar el conjunto de fotos** (el sintético que trae el cuaderno o uno propio con licencia), con **una carpeta por clase**.
2. **Entrenar un modelo con *transfer learning*** (reutilizar una red ya entrenada con millones de imágenes y adaptarla a vuestras clases).
3. **Entrenar una referencia «desde cero»** para ver cuánto aporta el *transfer learning*.
4. **Evaluar en el test** y hacer el **análisis de errores**: matriz de confusión y al menos **6 fotos mal clasificadas** con una hipótesis de por qué.
5. **Construir un demostrador**: una función que recibe una foto y devuelve clase, confianza y «revisar a mano» si la confianza es baja.
6. **Escribir el análisis de errores** de 1–2 páginas con una recomendación.

Qué vais a entregar: el modelo (`.keras` o cuaderno que lo reproduce), el demostrador y el análisis de errores.

Qué vais a aprender haciéndolo: a **reutilizar modelos preentrenados**, a **interpretar errores** de un modelo de visión y a decidir **qué se automatiza y qué se revisa a mano**.

Cuánto tiempo: 5 horas de proyecto (17 al 20/11).

Palabras que van a aparecer en este reto

Palabra	Qué significa, en sencillo
Visión artificial	IA que trabaja con imágenes
CNN	Red neuronal especializada en imágenes: detecta bordes, formas y objetos por capas
Transfer learning	Aprovechar una red ya entrenada (p. ej., con millones de fotos) y «reentrenar» solo su final para vuestro problema. Necesita menos fotos y menos tiempo
Congelar capas	Dejar sin cambiar las capas ya entrenadas y entrenar solo las nuevas
Fine-tuning (ajuste fino)	Después, descongelar algunas capas y ajustarlas con una tasa de aprendizaje muy baja
Clase	Cada categoría que debe reconocer el modelo (hotel, camping…)
Matriz de confusión	Tabla que muestra qué clases se confunden con cuáles
Métricas por clase	Precisión y exhaustividad (*recall*) de cada clase por separado
Confianza	Lo seguro que está el modelo de su respuesta (0–100 %)
Umbral de confianza	Por debajo de él, el modelo dice «revisar a mano»
Cobertura	Qué porcentaje de fotos decide el modelo solo (el resto va a revisión humana)
Licencia	Permiso de uso de una foto. Sin licencia clara, no se puede usar

Contexto del cliente. TurisData Canarias recibe cientos de fotos de alojamientos y zonas cada semana y las clasifica una persona a mano (hotel, apartamentos, casa rural, camping, piscina). Quiere saber si un modelo puede pre-clasificarlas y, sobre todo, **en qué se equivoca**, para decidir cuánto puede fiarse de él y qué fotos debe revisar una persona.

Qué se entrega y en qué formato

1. **Modelo** entrenado con *transfer learning* (archivo `.keras` o notebook que lo reproduce).
2. **Demostrador:** una función (o una pequeña interfaz en Colab) que recibe una foto y devuelve clase, confianza y «revisar a mano» si es baja.
3. **Análisis de errores** de 1–2 páginas (PDF o Markdown): métricas por clase, matriz de confusión y **al menos 6 ejemplos mal clasificados** con su hipótesis de causa.

Restricciones

- Dataset: el sintético que trae el notebook, o uno propio con **licencia clara** (fotos propias o con licencia abierta; guarda autor y licencia). Sin personas reconocibles.
- Estructura de carpetas: una por clase; mínimo 30 imágenes por clase (mejor 100 o más).
- Partición entrenamiento/validación/prueba fija; la prueba se usa una sola vez.
- Debe ejecutarse en CPU en pocos minutos; en Colab puedes usar MobileNetV2 con ImageNet (opcional).

Criterios de evaluación (0–4, la media alimenta «ejercicios y entregas»)

Criterio	Qué buscamos aquí
Corrección técnica	Carga y partición sin fugas, *transfer learning* bien aplicado (congelar, cabeza, *fine-tuning* con tasa baja), métricas por clase y comparación con una referencia (desde cero).
Reproducibilidad	Se ejecuta desde el README; semillas y versiones fijadas; script o instrucciones para obtener el dataset.
Análisis y comunicación al cliente	Errores analizados con ejemplos y causas; recomendación clara (qué se automatiza y qué se revisa a mano) con umbral de confianza justificado.
IA responsable	Riesgos concretos: representatividad del dataset, clases poco frecuentes, etiquetas dudosas, privacidad y licencias de las fotos; medidas aplicadas.

Pasos sugeridos

- **Día 1:** ejecuta `S11_04`, revisa la estructura del dataset (`contar_por_clase`) y decide si usarás el sintético o uno propio.

- **Día 2:** entrena el modelo con *transfer learning* y una referencia desde cero; mide en validación.
- **Día 3:** evalúa en prueba y haz el análisis de errores (matriz de confusión, ejemplos, hipótesis).
- **Día 4:** demostrador y umbral de confianza (cobertura frente a precisión).
- **Día 5:** informe y presentación de 5 minutos.

Guía paso a paso para principiantes

Paso 1 · Preparar los datos (día 1)

1. Abrid `S11_04_reto_vision_cliente` en Colab.
2. Decidid: ¿**dataset sintético** (lo genera `generar_dataset_sintetico.py`) o **uno propio**? - Si es propio: fotos tomadas por vosotros o con **licencia abierta**; guardad **autor y licencia**; **sin personas reconocibles**.
3. Estructura: **una carpeta por clase**, con **mínimo 30 imágenes por clase** (mejor 100 o más). Usad `contar_por_clase` para comprobarlo.
4. **Mirad las fotos** de cada clase. ¿Hay etiquetas dudosas? Anotadlas.
5. **Fijad la partición** entrenamiento / validación / prueba. La prueba se usa **una sola vez**.

Paso 2 · Entrenar con transfer learning y una referencia (día 2)

1. Cargad la red preentrenada (en Colab, MobileNetV2 con ImageNet es opcional; si no, la que trae el cuaderno).
2. **Congelad** las capas y entrenad solo la «cabeza» (las capas nuevas para vuestras clases).
3. *(Opcional)* **Fine-tuning** con una tasa de aprendizaje muy baja.
4. Entrenad también una **referencia «desde cero»** (una red pequeña sin preentrenar).
5. Medid ambas en **validación** y anotad las diferencias. Debería verse que el *transfer learning* aprende más rápido y con menos fotos.

Paso 3 · Evaluación y análisis de errores (día 3)

1. Evaluad en el **test** (una sola vez) y calculad **métricas por clase**.
2. Dibujad la **matriz de confusión**. ¿Qué clases se confunden más?
3. Sacad las fotos mal clasificadas y **elegid al menos 6**. Para cada una anotad:

Foto	Verdadera	Predicha	Hipótesis de causa	Mejora propuesta
(miniatura)	Camping	Casa rural	Hay una caseta de madera; poca luz	Más fotos de camping nocturnas

4. Buscad patrones: ¿son fotos oscuras? ¿clases poco frecuentes? ¿etiquetas dudosas? ¿fondos parecidos?

Paso 4 · Demostrador y umbral de confianza (día 4)

1. Escribid una función `clasificar(foto)` que devuelva **clase, confianza y una indicación «revisar a mano»** si la confianza es menor que un umbral.
2. **Elegid el umbral** midiendo en validación: con cada umbral, ¿qué % de fotos decide el modelo solo (*cobertura*) y qué % de esas acierta? Un umbral alto → menos fotos automáticas pero más fiables.
3. Probad con 2–3 fotos delante de vuestro equipo.

Paso 5 · Informe y presentación (día 5)

Informe de 1–2 páginas:

1. **Qué se ha hecho y con qué datos** (origen y licencia).
2. **Resultados:** tabla de métricas por clase, comparación con la referencia.
3. **Matriz de confusión interpretada.**
4. **Análisis de al menos 6 errores.**
5. **Recomendación al cliente:** qué clases se automatizan, qué fotos se revisan a mano y con qué umbral.
6. **Riesgos de IA responsable** (al menos dos) y medidas.

Cómo usar la IA en este reto

Para qué	Ejemplo de petición	Qué tenéis que comprobar
Entender el transfer learning	«Explícame transfer learning con la analogía de un cocinero experto que aprende un plato nuevo»	Que podáis contarlo vosotros
Interpretar una matriz	«Mi modelo confunde mucho camping con casa rural. ¿Qué causas habituales puede haber?»	Que las contrastáis mirando las **fotos reales**

Para qué	Ejemplo de petición	Qué tenéis que comprobar
Depurar código de Keras	«Me sale este error al cargar las imágenes: *[error]*»	Que entendéis la causa
Redactar hipótesis	«Dame ideas de por qué un modelo de imágenes confundiría fotos de piscina con hotel»	Que elijáis solo las que **se ven en vuestras fotos**

Reglas del reto:

- **Licencias y privacidad:** solo imágenes propias o con licencia abierta, con autor y licencia anotados. **Sin personas reconocibles.**
- La **prueba se usa una sola vez**.
- **Hipótesis con evidencia:** no basta decir «por falta de datos»; mirad las fotos y razonad.
- Declarad el uso de IA.

Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Fotos casi idénticas en train y test	Se parte sin cuidado	Partid antes de generar variantes de una misma foto
Pocas fotos en alguna clase	Es difícil conseguirlas	Mínimo 30 por clase; si no, avisad del límite en el informe
Fiarse solo de la exactitud global	Es un solo número	Mirad **métricas por clase** y la matriz de confusión
Analizar errores sin mirar las fotos	Es más cómodo	Mirad las fotos mal clasificadas una a una
Umbral de confianza «a ojo»	Se pone 0,5	Elegidlo midiendo cobertura y acierto
Usar fotos de internet sin licencia	«Total, es un curso»	Solo con licencia clara; guardad autor y licencia

Un buen resultado se parece a...

> *«El modelo con transfer learning alcanza un 91 % de acierto en test frente al 74 % de la referencia desde cero. Confunde camping con casa rural en 7 de 40 fotos, casi siempre con poca luz y casetas de madera. Recomendamos automatizar las fotos con confianza ≥ 0,85 (el 72 % de las fotos, con un 97 % de acierto) y revisar a mano el resto.»* (Cifras de ejemplo.)

Antes de entregar: autoevaluación rápida

Pregunta	Sí / No
¿El cuaderno se ejecuta de arriba abajo con semillas fijas?	
¿El dataset tiene carpeta por clase y hemos comprobado su contenido?	
¿Entrenamiento, validación y prueba están separados y la prueba se usó una vez?	
¿Hay un modelo con transfer learning y una referencia para compararlo?	
¿Hay métricas por clase y matriz de confusión interpretadas?	
¿Hay al menos 6 errores analizados con hipótesis y mejora?	
¿El demostrador devuelve clase, confianza y «revisar a mano»?	
¿Documentamos origen y licencia de los datos y al menos dos riesgos de IA responsable?	

Si os atascáis

- **Colab dice que no hay memoria:** reducid el tamaño de las imágenes o el número de fotos y reiniciad el entorno.
- **El modelo acierta siempre la misma clase:** hay desbalance; comprobad el número de fotos por clase.
- **No sabemos explicar un error:** mirad la foto como si fuerais el modelo: ¿qué rasgo la haría parecerse a la otra clase?
- Más ayuda: Cómo se trabaja un reto.

Qué se enseña en la *review*. El demostrador funcionando con 2–3 fotos, la matriz de confusión y tres errores comentados ("el modelo confunde X con Y porque… y propondríamos…").

Definition of Done

- [ ] El notebook se ejecuta de arriba abajo con semillas fijas.
- [ ] El dataset tiene estructura por carpetas y se ha comprobado su contenido por clase.
- [ ] Entrenamiento/validación/prueba separados; la prueba se usa una sola vez.
- [ ] Hay modelo con *transfer learning* y una referencia para compararlo.
- [ ] Métricas por clase y matriz de confusión interpretadas.
- [ ] Al menos 6 ejemplos mal clasificados analizados, con hipótesis y mejora propuesta.
- [ ] Demostrador con umbral de confianza y opción «revisar a mano».
- [ ] Se documentan origen y licencia de los datos y al menos dos riesgos de IA responsable.

Recursos

- Keras: guías «Transfer learning & fine-tuning» y «Image classification from scratch» (keras.io).
- Documentación de `keras.utils.image_dataset_from_directory`.
- Documentación de OpenCV para Python: `cv2.findContours`, `cv2.matchTemplate`.
- Guía de *Ultralytics YOLO* (docs.ultralytics.com), solo para la celda opcional de Colab.
- Creative Commons: cómo reconocer y citar licencias de imágenes (creativecommons.org).